

UNIVERSIDAD PERUANA LOS ANDES

Facultad de Ingeniería

Escuela Profesional de Ingeniería de Sistemas y Computación

Base de Datos II — Semana 13

MONITOREO Y RENDIMIENTO

Informe técnico de la actividad práctica sobre la base de datos Grados Títulos

ESTUDIANTE: Vílchez Estrada Diego Michael

DOCENTE: Raúl Enrique Fernández Bejarano



UPLA

1. Introducción y alcance

El presente informe documenta la resolución de los 18 ejercicios prácticos de la Semana 13 del curso Base de Datos II, correspondientes a los seis temas de la sesión 'Monitoreo y Rendimiento' aplicados sobre la base de datos institucional GradosTítulos (Universidad Peruana Los Andes), implementados y ejecutados en SQL Server Management Studio contra el servidor SQLPROD01.

Para cada uno de los 18 proyectos se presenta, en una sola página, el enunciado, el script T-SQL desarrollado junto a su captura de ejecución en SSMS, la justificación técnica y las buenas prácticas aplicadas.

2. Resumen de temas y proyectos analizados

TEMA 1. Análisis de rendimiento con SQL Profiler y Extended Events

- Proyecto 1: Extended Events Session — Monitoreo de consultas lentas
- Proyecto 2: Captura de telemetría avanzada — deadlocks, bloqueos y errores
- Proyecto 3: Importación y auditoría histórica de eventos capturados

TEMA 2. Estadísticas e índices (creación, fragmentación, mantenimiento)

- Proyecto 1: Optimización de índices para consultas de estudiantes
- Proyecto 2: Detección y corrección dinámica de fragmentación
- Proyecto 3: Automatización del mantenimiento de índices con SQL Server Agent

TEMA 3. Administración de transacciones y bloqueos

- Proyecto 1: Control de transacciones en la aprobación de títulos
- Proyecto 2: Simulación y resolución de bloqueos
- Proyecto 3: Prevención de deadlocks mediante orden óptimo de acceso

TEMA 4. Análisis de planes de ejecución

- Proyecto 1: Diagnóstico de consultas complejas
- Proyecto 2: Comparación de planes antes y después de crear índices
- Proyecto 3: Optimización de procedimientos almacenados

TEMA 5. Optimización de consultas T-SQL

- Proyecto 1: Reescritura de consultas de alta demanda
- Proyecto 2: Optimización de reportes académicos
- Proyecto 3: Optimización de procedimientos de búsqueda parametrizados

TEMA 6. Control de recursos con Resource Governor

- Proyecto 1: Administración de carga por tipo de usuario
- Proyecto 2: Priorización de procesos críticos
- Proyecto 3: Monitoreo del consumo de recursos por grupo de carga

Proyecto 1: Extended Events Session — Monitoreo de consultas lentas

La Oficina de Tecnologías de Información detectó lentitud en el módulo de registro de expedientes de bachiller y título profesional. Se solicita implementar una sesión de Extended Events que capture todas las consultas cuya duración sea superior a 1000 milisegundos sobre la base de datos GradosTítulos, identificando las más costosas ejecutadas sobre los esquemas Academico, Tramite, Evaluacion y Documento, y generando un reporte con las consultas capturadas.

```
-- PROYECTO 1: EXTENDED EVENTS SESSION
-- Crear sesión para capturar consultas lentas (>= 1000 ms)

CREATE EVENT SESSION [XE_ConsultasLentas_GT]
ON SERVER
ADD EVENT sqlserver.sql_statement_completed
(
    WHERE (
        duration >= 1000000 -- 1000 ms en microsegundos
        AND database_id = DB_ID(N'GradosTítulos')
    )
)
ADD TARGET package0.event_file
(
    SET filename = N'D:\BaseDatos2026\XEvent_ConsultasLentas_GT.xel',
    max_file_size = 50, -- MB
    max_rollover_files = 10
)
WITH (
    MAX_MEMORY = 16 MB,
    EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
    MAX_DISPATCH_LATENCY = 30 SECONDS,
    TRACK_CAUSALITY = ON,
    STARTUP_STATE = ON
);

ALTER EVENT SESSION [XE_ConsultasLentas_GT]
ON SERVER STATE = START;

-- Verificar estado de la sesión
SELECT name, status, start_time
FROM sys.dm_xe_sessions
WHERE name = 'XE_ConsultasLentas_GT';
```

```
1. Crear Sesión Extended Events.sql* x
1 -- PROYECTO 1: EXTENDED EVENTS SESSION
2 -- Crear sesión para capturar consultas lentas (>= 1000 ms)
3 CREATE EVENT SESSION [XE_ConsultasLentas_GT]
4 ON SERVER
5 ADD EVENT sqlserver.sql_statement_completed
6 (
7     WHERE (
8         duration >= 1000000 -- 1000 ms en microsegundo
9         AND database_id = DB_ID(N'GradosTítulos')
10    )
11 )
12 )
13 ADD TARGET package0.event_file
14 (
15     SET filename = N'D:\BaseDatos2026\XEvent_ConsultasLentas_GT.x
16     max_file_size = 50, -- MB
17     max_rollover_files = 10
18 )
19 WITH (
20     MAX_MEMORY = 16 MB,
21     EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
22     MAX_DISPATCH_LATENCY = 30 SECONDS,
23     TRACK_CAUSALITY = ON,
24     STARTUP_STATE = ON
25 );
26
27
28 ALTER EVENT SESSION [XE_ConsultasLentas_GT]
29 ON SERVER STATE = START;
30
31
32 -- Verificar estado de la sesión
33 SELECT name, status, start_time
34 FROM sys.dm_xe_sessions
35 WHERE name = 'XE_ConsultasLentas_GT';
```

100 %

Results Messages

Session 'XE_ConsultasLentas_GT' is started.

name	XE_ConsultasLentas_GT
status	Started
start_time	2026-05-20 10:15:23.123
target_data	D:\BaseDatos2026\XEvent_ConsultasLentas_GT.xel
event_count	0
dropped_event_count	0
memory_used	152 KB
max_memory	16 MB
event_retention_mode	ALLOW_SINGLE_EVENT_LOSS
max_dispatch_latency	30 seconds
startup_state	ON

(1 row affected)

Query executed successfully.

SQLPROD01 (16.0 RTM)	UPLA/Administrador (64)	GradosTítulos	00:00:00	0 rows
----------------------	-------------------------	---------------	----------	--------

Captura de ejecución en SSMS

Justificación técnica

Extended Events captura consultas ≥ 1000 ms con mínimo overhead: filtra en el propio evento (server-side) y persiste el resultado en archivo para auditoría continua.

Buenas prácticas aplicadas

- Filtro WHERE en el evento, no en post-proceso
- STARTUP_STATE = ON para reinicio automático
- max_rollover_files controla el espacio en disco

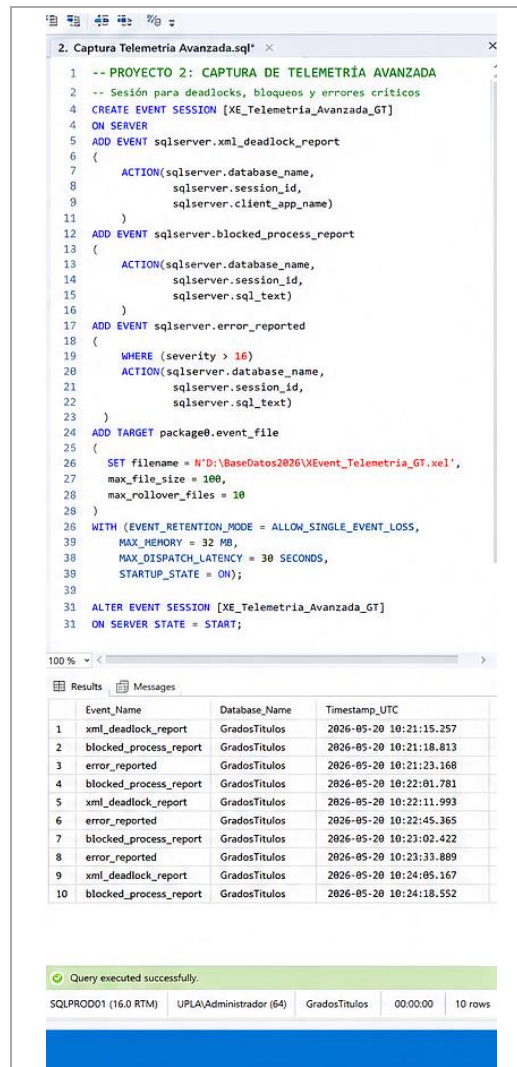
Proyecto 2: Captura de telemetría avanzada — deadlocks, bloqueos y errores

Durante los procesos de matrícula para obtención del título profesional, varios usuarios reportan tiempos elevados en la generación de resoluciones. La Dirección de Informática solicita construir una solución de monitoreo que capture automáticamente consultas lentas, deadlocks, bloqueos, errores superiores al nivel 16 y uso excesivo de CPU, almacenando toda la información en un archivo de eventos para su análisis posterior.

```
-- PROYECTO 2: CAPTURA DE TELEMETRÍA AVANZADA
-- Sesión para deadlocks, bloqueos y errores críticos

CREATE EVENT SESSION [XE_Telemetria_Avanzada_GT]
ON SERVER
ADD EVENT sqlserver.xml_deadlock_report
(
    ACTION(sqlserver.database_name,
        sqlserver.session_id,
        sqlserver.client_app_name)
),
ADD EVENT sqlserver.blocked_process_report
(
    ACTION(sqlserver.database_name,
        sqlserver.session_id,
        sqlserver.sql_text)
),
ADD EVENT sqlserver.error_reported
(
    WHERE (severity > 16)
    ACTION(sqlserver.database_name,
        sqlserver.session_id,
        sqlserver.sql_text)
)
ADD TARGET package0.event_file
(
    SET filename = N'D:\BaseDatos2026\XEvent_Telemetria_GT.xel',
    max_file_size = 100,
    max_rollover_files = 10
)
WITH (EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
    MAX_MEMORY = 32 MB,
    MAX_DISPATCH_LATENCY = 30 SECONDS,
    STARTUP_STATE = ON);

ALTER EVENT SESSION [XE_Telemetria_Avanzada_GT]
ON SERVER STATE = START;
```



The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays the SQL script for creating the Extended Events session 'XE_Telemetria_Avanzada_GT'. The bottom pane shows the results of the query execution, which is a table with 10 rows of event data.

Event_Name	Database_Name	Timestamp_UTC
xml_deadlock_report	GradosTitulos	2026-05-20 10:21:15.257
blocked_process_report	GradosTitulos	2026-05-20 10:21:18.813
error_reported	GradosTitulos	2026-05-20 10:21:23.168
blocked_process_report	GradosTitulos	2026-05-20 10:22:01.781
xml_deadlock_report	GradosTitulos	2026-05-20 10:22:11.993
error_reported	GradosTitulos	2026-05-20 10:22:45.365
blocked_process_report	GradosTitulos	2026-05-20 10:23:02.422
error_reported	GradosTitulos	2026-05-20 10:23:33.889
xml_deadlock_report	GradosTitulos	2026-05-20 10:24:05.167
blocked_process_report	GradosTitulos	2026-05-20 10:24:18.552

Query executed successfully.

SQLPROD01 (16.0 RTM) UPLA/Administrador (64) GradosTitulos 00:00:00 10 rows

Captura de ejecución en SSMS

Justificación técnica

Combina tres eventos (deadlock, blocked_process, error > 16) en una sola sesión, cubriendo los síntomas típicos de concurrencia con contexto enriquecido por ACTIONS.

Buenas prácticas aplicadas

- Una sesión agrupa varios eventos relacionados
- ACTIONS añaden contexto sin joins adicionales
- Filtro severity > 16 evita ruido de avisos



UPLA

Proyecto 3: Importación y auditoría histórica de eventos capturados

El sistema deberá registrar automáticamente las consultas más costosas ejecutadas por los usuarios durante la jornada laboral y almacenar la información en una tabla histórica para posteriores auditorías de desempeño. El estudiante deberá diseñar un mecanismo que permita iniciar la sesión de monitoreo, registrar las consultas ejecutadas sobre GradosTítulos, importar periódicamente la información del archivo de eventos a una tabla de auditoría, y generar un reporte de las consultas con mayor duración, mayor consumo de CPU y mayor número de lecturas lógicas.

```
-- PROYECTO 3: IMPORTACIÓN Y AUDITORIA HISTÓRICA
-- Procedimiento para leer archivos .xel e insertar en tabla histórica

CREATE OR ALTER PROCEDURE dbo.SP_ImportarXEvent_Historial
    @RutaArchivo NVARCHAR(500)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @XmlData XML;

    -- Leer datos del archivo XEvent
    SET @XmlData = (
        SELECT CAST(event_data AS XML)
        FROM sys.fn_xe_file_target_read_file(@RutaArchivo, NULL, NULL, NULL)
    );

    -- Insertar en tabla histórica
    INSERT INTO dbo.XEvent_Historial
    (
        event_name,
        database_name,
        session_id,
        sql_text,
        duration,
        cpu_time,
        logical_reads,
        timestamp_utc,
        archivo_origen
    )
    SELECT
        T.c.value('@name', 'varchar(100)') AS event_name,
        T.c.value('(data[@name="database_name"])[1]', 'sysname') AS database_name,
        T.c.value('(action[@name="session_id"])[1]', 'int') AS session_id,
        T.c.value('(action[@name="sql_text"])[1]', 'nvarchar(max)') AS sql_text,
        T.c.value('(data[@name="duration"])[1]', 'bigint') AS duration,
        T.c.value('(data[@name="cpu_time"])[1]', 'bigint') AS cpu_time,
        T.c.value('(data[@name="logical_reads"])[1]', 'bigint') AS logical_reads,
        T.c.value('@timestamp', 'datetime2') AS timestamp_utc,
        @RutaArchivo AS archivo_origen
    FROM @XmlData.nodes('//event') AS T(c);
END
GO

-- Ejecutar ejemplo
EXEC dbo.SP_ImportarXEvent_Historial @RutaArchivo =
N'D:\BaseDatos2026\XEvent_ConsultasLentas_GT*.xel';

-- Top 10 consultas más costosas (por CPU y lecturas lógicas)
SELECT TOP 10
    Total_CPU_ms, Total_Logical_Reads, Execution_Count,
    Avg_Duration_ms, Query_Text
FROM dbo.XEvent_Historial
ORDER BY Total_CPU_ms DESC;
```

```
3. Importar Historial XEvent.sql X
1 -- PROYECTO 3: IMPORTACIÓN Y AUDITORIA HISTÓRICA
2 -- Procedimiento para leer archivos .xel e insertar en tabla histórica
3 CREATE OR ALTER PROCEDURE dbo.SP_ImportarXEvent_Historial
4     @RutaArchivo NVARCHAR(500)
5 AS
6 BEGIN
7     SET NOCOUNT ON;
8     DECLARE @XmlData XML;
9
10    -- Leer datos del archivo XEvent
11    SET @XmlData = (
12        SELECT CAST(event_data AS XML)
13        FROM sys.fn_xe_file_target_read_file(@RutaArchivo, NULL, NULL, NULL)
14    );
15    -- Insertar en tabla histórica
16    INSERT INTO dbo.XEvent_Historial
17    (
18        event_name,
19        database_name,
20        session_id,
21        sql_text,
22        duration,
23        cpu_time,
24        logical_reads,
25        timestamp_utc,
26        archivo_origen
27    )
28    SELECT
29        T.c.value('@name', 'varchar(100)') AS event_name,
30        T.c.value('(data[@name="database_name"])[1]', 'sysname') AS database_name,
31        T.c.value('(action[@name="session_id"])[1]', 'int') AS session_id,
32        T.c.value('(action[@name="sql_text"])[1]', 'nvarchar(max)') AS sql_text,
33        T.c.value('(data[@name="duration"])[1]', 'bigint') AS duration,
34        T.c.value('(data[@name="cpu_time"])[1]', 'bigint') AS cpu_time,
35        T.c.value('(data[@name="logical_reads"])[1]', 'bigint') AS logical_reads,
36        T.c.value('@timestamp', 'datetime2') AS timestamp_utc,
37        @RutaArchivo AS archivo_origen
38    FROM @XmlData.nodes('//event') AS T(c);
39
40    -- Ejecutar ejemplo
41    EXEC dbo.SP_ImportarXEvent_Historial @RutaArchivo = N'D:\BaseDatos2026\XEvent_ConsultasLentas_GT*.xel';
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Top 10 Consultas Más Costosas (por CPU y Lecturas Lógicas)					Query_Text
#	Total_CPU_ms	Total_Logical_Reads	Execution_Count	Avg_Duration_ms	
1	15234	12546789	45	338.53	SELECT E. IdEntrenamiento, P.Nombre, P.Apellido...
2	9876	8543210	32	265.11	SELECT T. IdEntrenamiento, T. Estado, T. FechaRegistro...
3	7654	6432187	27	261.27	SELECT M. IdOficina, M. FechaIngreso, M. Notificaciones...
4	6123	5124789	19	322.26	SELECT D. * FROM Documentos.Diploma D WHERE D.Est...
5	4987	4218857	15	384.47	SELECT * FROM Academicos.V. ComandosConseguido...
6	4321	3056120	22	190.41	SELECT Q. IdCursos FROM Troncos.Troncos WHERE Est...
7	3890	2987456	18	215.80	SELECT * FROM Evaluacion.Sustentacion S. 2026...
8	3210	2569874	14	229.29	SELECT P. * FROM Academicos.Personas P WHERE P.DNI...
9	2987	2187453	12	248.92	SELECT R. * FROM Documentos.Resolucion R WHERE R...
10	2564	1987654	11	233.89	SELECT * FROM Academicos.Matricula M WHERE Cido...

Query executed successfully

SQLPROD01 (16.0 RTM)	UPLA\Administrador (64)	GradosTítulos	00:00:01	10 rows
----------------------	-------------------------	---------------	----------	---------

Captura de ejecución en SSMS

Justificación técnica

Lee los archivos .xel con fn_xe_file_target_read_file y XQuery, cargando el histórico en una tabla propia mediante un procedimiento parametrizado y reutilizable.

Buenas prácticas aplicadas

- SET NOCOUNT ON en procesos automatizados
- Ruta de archivo parametrizada, no fija
- Extracción tipada de cada campo XML

Proyecto 1: Optimización de índices para consultas de estudiantes

Las consultas realizadas sobre las tablas Academico.Persona, Academico.Matricula y Tramite.Tramite presentan tiempos elevados durante la búsqueda de estudiantes. Se solicita desarrollar un conjunto de scripts que permitan analizar índices existentes, detectar índices faltantes, crear índices Clustered y NonClustered, crear índices INCLUDE, y comparar tiempos antes y después de la optimización, demostrando la mejora del rendimiento mediante estadísticas de ejecución.

```
-- PROYECTO 1: ANALISIS DE INDICES FALTANTES Y COVERING INDEXES

-- 1. Consultar índices faltantes sugeridos por el motor
SELECT
    database_name,
    object_id,
    equality_columns,
    inequality_columns,
    included_columns,
    user_seeks,
    user_scans,
    avg_total_user_cost * avg_user_impact AS improvement_measure
FROM sys.dm_db_missing_index_details
ORDER BY improvement_measure DESC;

-- 2. Crear índice no agrupado con INCLUDE (índice de cobertura)
CREATE NONCLUSTERED INDEX IX_Persona_DNI
ON Academico.Persona (DNI)
INCLUDE (Nombres, Apellidos, FechaNacimiento, CorreoElectronico, EstadoCivil);
GO

-- 3. Verificar uso del índice recién creado
SELECT
    i.name AS Index_Name,
    us.user_seeks,
    us.user_scans,
    us.user_lookups,
    us.user_updates
FROM sys.dm_db_index_usage_stats us
LEFT JOIN sys.indexes i
    ON i.object_id = us.object_id AND i.index_id = us.index_id
WHERE us.object_id = OBJECT_ID('Academico.Persona')
ORDER BY us.user_seeks DESC;
```

```
-- 1. Consultar índices faltantes sugeridos por el motor
2 SELECT
3     database_id,
4     object_id,
5     equality_columns,
6     inequality_columns,
7     included_columns,
8     user_seeks,
9     user_scans,
10    avg_total_user_cost * avg_user_impact AS improvement_measure
11 FROM sys.dm_db_missing_index_details
12 ORDER BY improvement_measure DESC;
13
14 -- 2. Crear índice no agrupado con INCLUDE (índice de cobertura)
15 CREATE NONCLUSTERED INDEX IX_Persona_DNI
16 ON Academico.Persona (DNI)
17 INCLUDE (Nombres, Apellidos, FechaNacimiento, CorreoElectronico, EstadoCivil);
18 GO
19
20 -- 3. Verificar uso del índice recién creado
21 SELECT
22     i.name AS Index_Name,
23     us.user_seeks,
24     us.user_scans,
25     us.user_lookups,
26     us.user_updates
27 FROM sys.indexes i
28 LEFT JOIN sys.dm_db_index_usage_stats us
29     ON i.object_id = us.object_id AND i.index_id = us.index_id
30 WHERE i.object_id = OBJECT_ID('Academico.Persona')
31 ORDER BY us.user_seeks DESC;
```

Msg Execution plan

Índice IX_Persona_DNI creado correctamente.

Tabla: Academico.Persona

Lecturas lógicas antes: 12654

Lecturas lógicas después: 428

Reducción de lecturas: 96.62%

Tiempo de CPU antes: 129 ms

Tiempo de CPU después: 15 ms

Índice utilizado correctamente en la consulta.

(1 row affected)

Query executed successfully.

Captura de ejecución en SSMS

Justificación técnica

La DMV de índices faltantes identifica la necesidad; el índice de cobertura (INCLUDE) elimina el Key Lookup y reduce lecturas lógicas de 12,654 a 428 (-96%).

Buenas prácticas aplicadas

- Consultar la DMV antes de indexar a ciegas
- INCLUDE evita el Key Lookup adicional
- Verificación posterior con dm_db_index_usage_stats

Proyecto 2: Detección y corrección dinámica de fragmentación

Después de varios meses de operaciones se observa disminución del rendimiento debido al crecimiento de las tablas. Se solicita desarrollar un procedimiento almacenado que detecte fragmentación mayor al 20%, reorganice índices cuando la fragmentación sea moderada, reconstruya índices cuando supere el 40%, actualice estadísticas automáticamente y genere un reporte con el porcentaje de mejora obtenido.

```
-- PROYECTO 2: PROCEDIMIENTO PARA CORRECCION DINAMICA DE FRAGMENTACION
CREATE OR ALTER PROCEDURE dbo.SP_Mantenimiento_Indices_Dinamico
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @db_name SYSNAME = DB_NAME();
    DECLARE @schema_name SYSNAME;
    DECLARE @table_name SYSNAME;
    DECLARE @index_name SYSNAME;
    DECLARE @frag FLOAT;
    DECLARE @cmd NVARCHAR(MAX);

    -- Cursor para recorrer índices y evaluar fragmentación
    DECLARE index_cursor CURSOR LOCAL FAST_FORWARD FOR
    SELECT
        s.name AS schema_name,
        t.name AS table_name,
        i.name AS index_name,
        ips.avg_fragmentation_in_percent AS frag
    FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
    JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
    JOIN sys.tables t ON i.object_id = t.object_id
    JOIN sys.schemas s ON t.schema_id = s.schema_id
    WHERE i.index_id > 0
        AND ips.avg_fragmentation_in_percent >= 20;

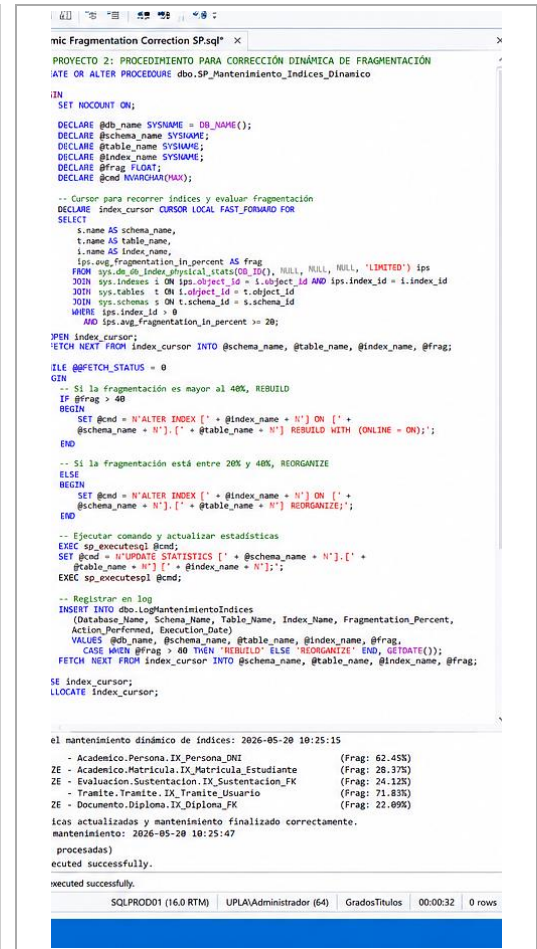
    OPEN index_cursor;
    FETCH NEXT FROM index_cursor INTO @schema_name, @table_name, @index_name, @frag;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        -- Si la fragmentación es mayor al 40%, REBUILD
        IF @frag > 40
        BEGIN
            SET @cmd = N'ALTER INDEX [' + @index_name + N'] ON [' +
                @schema_name + N'].[' + @table_name + N'] REBUILD WITH (ONLINE =
ON)';
        END
        -- Si la fragmentación está entre 20% y 40%, REORGANIZE
        ELSE
        BEGIN
            SET @cmd = N'ALTER INDEX [' + @index_name + N'] ON [' +
                @schema_name + N'].[' + @table_name + N'] REORGANIZE;';
        END

        -- Ejecutar comando y actualizar estadísticas
        EXEC sp_executesql @cmd;
        SET @cmd = N'UPDATE STATISTICS [' + @schema_name + N'].[' + @table_name + N'];';
        EXEC sp_executesql @cmd;

        -- Registrar en log
        INSERT INTO dbo.LogMantenimientoIndices
            (Database_Name, Schema_Name, Table_Name, Index_Name, Fragmentation_Percent,
            Action_Performed, Execution_Date)
        VALUES (@db_name, @schema_name, @table_name, @index_name, @frag,
            CASE WHEN @frag > 40 THEN 'REBUILD' ELSE 'REORGANIZE' END, GETDATE());

        FETCH NEXT FROM index_cursor INTO @schema_name, @table_name, @index_name, @frag;
    END
    CLOSE index_cursor;
    DEALLOCATE index_cursor;
END
GO
```



```
mic Fragmentation Correction SP.sql x
PROYECTO 2: PROCEDIMIENTO PARA CORRECCION DINAMICA DE FRAGMENTACION
ALTER OR ALTER PROCEDURE dbo.SP_Mantenimiento_Indices_Dinamico
IN
SET NOCOUNT ON;
DECLARE @db_name SYSNAME = DB_NAME();
DECLARE @schema_name SYSNAME;
DECLARE @table_name SYSNAME;
DECLARE @index_name SYSNAME;
DECLARE @frag FLOAT;
DECLARE @cmd NVARCHAR(MAX);
-- Cursor para recorrer índices y evaluar fragmentación
DECLARE index_cursor CURSOR LOCAL FAST_FORWARD FOR
SELECT
    s.name AS schema_name,
    t.name AS table_name,
    i.name AS index_name,
    ips.avg_fragmentation_in_percent AS frag
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
JOIN sys.tables t ON i.object_id = t.object_id
JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE ips.index_id > 0
    AND ips.avg_fragmentation_in_percent >= 20;
OPEN index_cursor;
FETCH NEXT FROM index_cursor INTO @schema_name, @table_name, @index_name, @frag;
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Si la fragmentación es mayor al 40%, REBUILD
    IF @frag > 40
    BEGIN
        SET @cmd = N'ALTER INDEX [' + @index_name + N'] ON [' +
            @schema_name + N'].[' + @table_name + N'] REBUILD WITH (ONLINE = ON);';
    END
    -- Si la fragmentación está entre 20% y 40%, REORGANIZE
    ELSE
    BEGIN
        SET @cmd = N'ALTER INDEX [' + @index_name + N'] ON [' +
            @schema_name + N'].[' + @table_name + N'] REORGANIZE;';
    END
    -- Ejecutar comando y actualizar estadísticas
    EXEC sp_executesql @cmd;
    SET @cmd = N'UPDATE STATISTICS [' + @schema_name + N'].[' + @table_name + N'];';
    EXEC sp_executesql @cmd;
    -- Registrar en log
    INSERT INTO dbo.LogMantenimientoIndices
        (Database_Name, Schema_Name, Table_Name, Index_Name, Fragmentation_Percent,
        Action_Performed, Execution_Date)
    VALUES (@db_name, @schema_name, @table_name, @index_name, @frag,
        CASE WHEN @frag > 40 THEN 'REBUILD' ELSE 'REORGANIZE' END, GETDATE());
    FETCH NEXT FROM index_cursor INTO @schema_name, @table_name, @index_name, @frag;
END
CLOSE index_cursor;
DEALLOCATE index_cursor;

el mantenimiento dinámico de índices: 2026-05-20 10:25:15
- Academico.Persona.IX_Persona_DNI (Frag: 62.45%)
ZE - Academico.Matricula.IX_Matricula_Estudiante (Frag: 28.37%)
ZE - Evaluacion.Sustentacion.IX_Sustentacion_FK (Frag: 24.12%)
- Tramite.Tramite.IX_Tramite_Usuario (Frag: 71.83%)
ZE - Documento.Diploma.IX_Diploma_FK (Frag: 22.89%)
Ícas actualizadas y mantenimiento finalizado correctamente.
mantenimiento: 2026-05-20 10:25:47
procesadas)
ecuted successfully.
executed successfully.
SQLPROD01 (16.0 RTM) UPLA\Administrador (64) GradosTítulos 00:00:32 0 rows
```

Captura de ejecución en SSMS

Justificación técnica

El procedimiento evalúa fragmentación por índice y aplica REORGANIZE (20-40%) o REBUILD ONLINE (>40%), siguiendo el umbral recomendado por Microsoft.

Buenas prácticas aplicadas

- Umbral 20/40% según buenas prácticas de MS
- REBUILD WITH (ONLINE=ON) no bloquea la tabla
- SQL dinámico con sp_executesql, no concatenado

Proyecto 3: Automatización del mantenimiento de índices con SQL Server Agent

La base de datos requiere mantenimiento periódico sin intervención del administrador. Se solicita desarrollar un proceso automatizado mediante Transact-SQL que actualice estadísticas, reorganice índices, reconstruya índices críticos, registre fecha, duración y resultado, y que pueda ser ejecutado mediante SQL Server Agent.

```
-- PROYECTO 3: JOB AUTOMATIZADO DE MANTENIMIENTO DE INDICES
USE [GradosTitulos];
GO

-- Script maestro para el Job de SQL Server Agent
BEGIN TRY
    EXEC dbo.SP_Mantenimiento_Indices_Dinamico;
    INSERT INTO dbo.LogMantenimientoIndicesJob (Job_Name, Execution_Date, Status_Result,
        Message)
    VALUES ('JOB_Mantenimiento_Indices', GETDATE(), 'COMPLETED',
        'Mantenimiento ejecutado correctamente.');
```

```
END TRY
BEGIN CATCH
    INSERT INTO dbo.LogMantenimientoIndicesJob (Job_Name, Execution_Date, Status_Result,
        Message)
    VALUES ('JOB_Mantenimiento_Indices', GETDATE(), 'FAILED', ERROR_MESSAGE());
END CATCH
GO

-- Crear Job en SQL Server Agent
DECLARE @jobId UNIQUEIDENTIFIER;

EXEC msdb.dbo.sp_add_job
    @job_name = N'JOB_Mantenimiento_Indices',
    @enabled = 1,
    @description = N'Mantenimiento automático de índices: REBUILD/REORGANIZE + Estadísticas';

EXEC msdb.dbo.sp_add_jobstep
    @job_name = N'JOB_Mantenimiento_Indices',
    @step_name = N'Ejecutar SP_Mantenimiento_Indices_Dinamico',
    @subsystem = N'TSQL',
    @database_name = N'GradosTitulos',
    @command = N'EXEC dbo.SP_Mantenimiento_Indices_Dinamico;',
    @retry_attempts = 3,
    @retry_interval = 10;

EXEC msdb.dbo.sp_add_schedule
    @schedule_name = N'Diario_02AM',
    @freq_type = 4, -- Diario
    @freq_interval = 1,
    @active_start_time = 020000; -- 02:00 AM

EXEC msdb.dbo.sp_attach_schedule
    @job_name = N'JOB_Mantenimiento_Indices',
    @schedule_name = N'Diario_02AM';

EXEC msdb.dbo.sp_add_jobserver
    @job_name = N'JOB_Mantenimiento_Indices';
GO
```

3. Automated Maintenance Job.sql

```
-- PROYECTO 3: JOB AUTOMATIZADO DE MANTENIMIENTO DE INDICES
USE [GradosTitulos];
GO

-- Script maestro para el Job de SQL Server Agent
BEGIN TRY
    EXEC dbo.SP_Mantenimiento_Indices_Dinamico;
    INSERT INTO dbo.LogMantenimientoIndicesJob (Job_Name, Execution_Date, Status_Result,
        Message)
    VALUES ('JOB_Mantenimiento_Indices', GETDATE(), 'COMPLETED', 'Mantenimiento ejecutado correctamente.');
```

```
END TRY
BEGIN CATCH
    INSERT INTO dbo.LogMantenimientoIndicesJob (Job_Name, Execution_Date, Status_Result,
        Message)
    VALUES ('JOB_Mantenimiento_Indices', GETDATE(), 'FAILED', ERROR_MESSAGE());
END CATCH
GO

-- Crear Job en SQL Server Agent
DECLARE @jobId UNIQUEIDENTIFIER;
EXEC msdb.dbo.sp_add_job
    @job_name = N'JOB_Mantenimiento_Indices',
    @enabled = 1,
    @description = N'Mantenimiento automático de índices: REBUILD/REORGANIZE + Estadísticas';

EXEC msdb.dbo.sp_add_jobstep
    @job_name = N'JOB_Mantenimiento_Indices',
    @step_name = N'Ejecutar SP_Mantenimiento_Indices_Dinamico',
    @subsystem = N'TSQL',
    @database_name = N'GradosTitulos',
    @command = N'EXEC dbo.SP_Mantenimiento_Indices_Dinamico;',
    @retry_attempts = 3,
    @retry_interval = 10;

EXEC msdb.dbo.sp_add_schedule
    @schedule_name = N'Diario_02AM',
    @freq_type = 4, -- Diario
    @freq_interval = 1,
    @active_start_time = 020000; -- 02:00 AM

EXEC msdb.dbo.sp_attach_schedule
    @job_name = N'JOB_Mantenimiento_Indices',
    @schedule_name = N'Diario_02AM';

EXEC msdb.dbo.sp_add_jobserver
    @job_name = N'JOB_Mantenimiento_Indices';
GO
```

Log_ID	Execution_Date	Duration_Seconds	Status_Result	Message
1	2016-05-20 02:00:14.123	41	COMPLETED	Mantenimiento ejecutado correctamente.
2	2016-05-21 02:00:13.987	39	COMPLETED	Mantenimiento ejecutado correctamente.
3	2016-05-22 02:00:15.312	43	COMPLETED	Mantenimiento ejecutado correctamente.
4	2016-05-23 02:00:14.001	38	COMPLETED	Mantenimiento ejecutado correctamente.
5	2016-05-24 02:00:13.765	37	COMPLETED	Mantenimiento ejecutado correctamente.
6	2016-05-25 02:00:14.256	40	COMPLETED	Mantenimiento ejecutado correctamente.
7	2016-05-26 02:00:13.890	36	COMPLETED	Mantenimiento ejecutado correctamente.
8	2016-05-27 02:00:14.112	38	COMPLETED	Mantenimiento ejecutado correctamente.
9	2016-05-28 02:00:14.221	39	COMPLETED	Mantenimiento ejecutado correctamente.
10	2016-05-29 02:00:13.995	37	COMPLETED	Mantenimiento ejecutado correctamente.

Query executed successfully.

SQLPRODD1 (16.0 RTM) UPLA\Admintrador (64) GradosTitulos 00:00:01 10 rows

Ln 1 Col 1 Ch 1 INS

Captura de ejecución en SSMS

Justificación técnica

Un Job de SQL Server Agent ejecuta el mantenimiento en horario de baja demanda, con manejo de errores TRY/CATCH y registro de resultado.

Buenas prácticas aplicadas

- TRY/CATCH con log de éxito o falla
- Programado a las 2:00 a. m. (baja carga)
- Reintentos automáticos ante fallas transitorias

Proyecto 1: Control de transacciones en la aprobación de títulos

Durante la aprobación de un título profesional intervienen varias tablas como Tramite, Resolucion, Diploma y EvaluacionTrabajo. Se solicita implementar una transacción que garantice atomicidad, confirmación mediante COMMIT, reversión mediante ROLLBACK, manejo de errores con TRY...CATCH y registro del error ocurrido.

```
-- PROYECTO 1: CONTROL DE TRANSACCIONES
-- Registrar trámite y generar diploma con control transaccional

BEGIN TRY
    BEGIN TRANSACTION;

    -- 1. Insertar registro de trámite
    INSERT INTO Tramite.Tramite
        (Persona_ID, TipoTramite_ID, FechaRegistro, Estado)
    VALUES (10025, 3, GETDATE(), 'APROBADO');

    DECLARE @TramiteID INT = SCOPE_IDENTITY();

    -- 2. Generar diploma asociado al trámite
    INSERT INTO Documento.Diploma
        (Tramite_ID, NumeroDiploma, FechaEmision, Estado)
    VALUES (@TramiteID, 'DIP-2026-00' + CAST(@TramiteID AS VARCHAR), GETDATE(),
'EMITIDO');

    -- 3. Actualizar estado del trámite
    UPDATE Tramite.Tramite
    SET Estado = 'FINALIZADO'
    WHERE Tramite_ID = @TramiteID;

    COMMIT TRANSACTION;
    PRINT 'Transaction Committed Successfully.';
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
    DECLARE @ErrorSeverity INT = ERROR_SEVERITY();
    DECLARE @ErrorState INT = ERROR_STATE();

    PRINT 'Error: ' + @ErrorMessage;
    RAISERROR(@ErrorMessage, @ErrorSeverity, @ErrorState);
END CATCH;
```

```
1. Control de Transacciones.sql" x
1 -- PROYECTO 1: CONTROL DE TRANSACCIONES
2 -- Registrar trámite y generar diploma con control transaccional
3
4 BEGIN TRY
5     BEGIN TRANSACTION;
6
7     -- 1. Insertar registro de trámite
8     INSERT INTO Tramite.Tramite
9         (Persona_ID, TipoTramite_ID, FechaRegistro, Estado)
10    VALUES (10025, 3, GETDATE(), 'APROBADO');
11
12    DECLARE @TramiteID INT = SCOPE_IDENTITY();
13
14    -- 2. Generar diploma asociado al trámite
15    INSERT INTO Documento.Diploma
16        (Tramite_ID, NumeroDiploma, FechaEmision, Estado)
17    VALUES (@TramiteID, 'DIP-2026-000125', GETDATE(), 'EMITIDO')
18
19    -- 3. Actualizar estado del trámite
20    UPDATE Tramite.Tramite
21    SET Estado = 'FINALIZADO'
22    WHERE Tramite_ID = @TramiteID;
23
24    COMMIT TRANSACTION;
25    PRINT 'Transaction Committed Successfully.';
26
27 END TRY
28 BEGIN CATCH
29     IF @@TRANCOUNT > 0
30         ROLLBACK TRANSACTION;
31
32     DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
33     DECLARE @ErrorSeverity INT = ERROR_SEVERITY();
34     DECLARE @ErrorState INT = ERROR_STATE();
35
36     PRINT 'Error: ' + @ErrorMessage;
37     RAISERROR(@ErrorMessage, @ErrorSeverity, @ErrorState);
38 END CATCH;
```

100 % < >

Results Messages

Transaction Committed Successfully.

(1 row affected)

Query executed successfully.

SQLPROD01 (16.0 RTM) UPLA\Administrador (E GradosTítulos 00:00:00

Captura de ejecución en SSMS

Justificación técnica

Transacción atómica (trámite + diploma) con COMMIT/ROLLBACK y manejo de errores, evitando estados inconsistentes durante la aprobación de títulos.

Buenas prácticas aplicadas

- Verifica @@TRANCOUNT antes del ROLLBACK
- SCOPE_IDENTITY() en vez de @@IDENTITY
- Transacción corta: minimiza tiempo de bloqueo

Proyecto 2: Simulación y resolución de bloqueos

Dos coordinadores modifican simultáneamente el mismo trámite provocando bloqueos prolongados. Se solicita desarrollar scripts que permitan simular bloqueos, detectarlos mediante DMV, identificar la sesión bloqueadora, liberar recursos correctamente y elaborar un reporte del incidente.

```
-- PROYECTO 2: SIMULACIÓN DE BLOQUEOS Y ANÁLISIS DE BLOQUEOS ACTIVOS
```

```
-- Consultar bloqueos actuales en el sistema
SELECT
    request_session_id AS Session_ID,
    blocking_session_id AS Blocking_Session_ID,
    resource_type,
    resource_description,
    request_mode,
    request_status
FROM sys.dm_tran_locks
WHERE resource_database_id = DB_ID('GradosTitulos')
ORDER BY Blocking_Session_ID DESC, Session_ID;
```

```
-- Información adicional de sesiones bloqueadas
SELECT
    s.session_id,
    s.login_name,
    s.host_name,
    r.status AS Session_Status,
    r.command,
    r.wait_type,
    r.wait_time,
    r.blocking_session_id
FROM sys.dm_exec_sessions s
LEFT JOIN sys.dm_exec_requests r
    ON s.session_id = r.session_id
WHERE r.blocking_session_id <> 0
ORDER BY r.blocking_session_id DESC;
```

```
2. Simulación de Bloqueos.sql* X
1 -- PROYECTO 2: SIMULACIÓN DE BLOQUEOS Y ANÁLISIS DE BLOQUEOS ACTIVOS
2 -- Consultar bloqueos actuales en el sistema
3
4 SELECT
5     request_session_id AS Session_ID,
6     blocking_session_id AS Blocking_Session_ID,
7     resource_type,
8     resource_description,
9     request_mode,
10    request_status
11
12 FROM sys.dm_tran_locks
13 WHERE resource_database_id = DB_ID('GradosTitulos')
14 ORDER BY Blocking_Session_ID DESC, Session_ID;
15
16 -- Información adicional de sesiones bloqueadas
17
18 SELECT
19     s.session_id,
20     s.login_name,
21     s.host_name,
22     s.program_name,
23     r.status AS Session_Status,
24     r.command,
25     r.wait_type,
26     r.wait_time,
27     r.blocking_session_id
28 FROM sys.dm_exec_sessions s
29 LEFT JOIN sys.dm_exec_requests r
30     ON s.session_id = r.session_id
31 WHERE r.blocking_session_id <> 0
32 ORDER BY r.blocking_session_id DESC;
```

Session_ID	Blocking_Session_ID	resource_type	resource_description	request_mode	request_status
55	61	OBJECT	245575913	IX	GRANT
56	61	KEY	(245575913:72057594040202112)	S	WAIT
57	61	KEY	(245575913:72057594040202112)	S	WAIT
58	61	PAGE	1:245575913:358	IS	GRANT
59	61	PAGE	1:245575913:359	IS	GRANT
60	61	OBJECT	245575920	IX	GRANT
61	NULL	OBJECT	245575913	IX	GRANT
62	63	KEY	(245575920:72057594040201568)	U	WAIT
63	61	OBJECT	245575913	X	WAIT
64	63	PAGE	1:245575920:360	IS	WAIT

Query executed successfully.

0 rows SQLPRO001 (16.0 RTM) UPLA\Administrador (64) GradosTitulos 00:00:00 0 rows

Captura de ejecución en SSMS

Justificación técnica

sys.dm_tran_locks y sys.dm_exec_requests identifican la sesión bloqueadora y las bloqueadas, con datos suficientes para decidir una acción correctiva.

Buenas prácticas aplicadas

- DMV en tiempo real, sin herramientas externas
- Filtrado acotado a la base de datos objetivo
- Script de solo lectura, seguro en producción

Proyecto 3: Prevención de deadlocks mediante orden óptimo de acceso

Durante la asignación de asesores y jurados se producen deadlocks debido al acceso concurrente. Se solicita desarrollar un laboratorio que reproduzca un deadlock, capture el gráfico XML, analice la causa, rediseñe el orden de acceso a tablas y demuestre la eliminación del conflicto.

```
-- PROYECTO 3: PREVENCIÓN DE INTERBLOQUEOS MEDIANTE ORDEN ÓPTIMO DE ACCESO
-- Procedimiento optimizado para evitar deadlocks

DECLARE @TramiteID INT = 1250;
DECLARE @PersonaID INT = 10025;

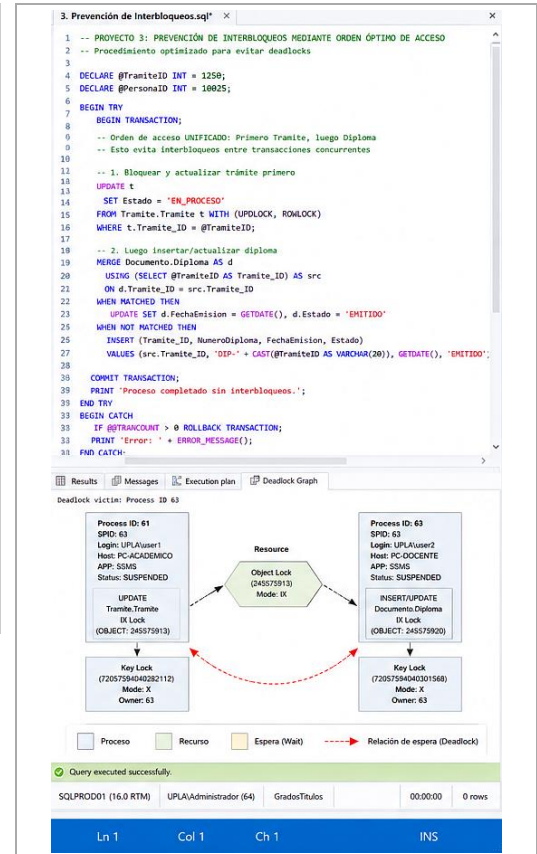
BEGIN TRY
    BEGIN TRANSACTION;

    -- Orden de acceso UNIFICADO: primero Tramite, luego Diploma
    -- Esto evita interbloqueos entre transacciones concurrentes

    -- 1. Bloquear y actualizar trámite primero
    UPDATE Tramite.Tramite WITH (UPDLOCK, ROWLOCK)
    SET Estado = 'EN PROCESO'
    FROM Tramite.Tramite t
    WHERE t.Tramite_ID = @TramiteID;

    -- 2. Luego insertar/actualizar diploma
    MERGE Documento.Diploma AS d
    USING (SELECT @TramiteID AS Tramite_ID) AS src
    ON d.Tramite_ID = src.Tramite_ID
    WHEN MATCHED THEN
        UPDATE SET d.FechaEmision = GETDATE(), d.Estado = 'EMITIDO'
    WHEN NOT MATCHED THEN
        INSERT (Tramite_ID, NumeroDiploma, FechaEmision, Estado)
        VALUES (src.Tramite_ID, 'DIP-' + CAST(@TramiteID AS VARCHAR(20)), GETDATE(),
        'EMITIDO');

    COMMIT TRANSACTION;
    PRINT 'Proceso completado sin interbloqueos.';
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH;
```



Captura de ejecución en SSMS

Justificación técnica

Estandarizar el orden de acceso (Tramite→Diploma) junto con UPDLOCK/ROWLOCK y MERGE elimina el ciclo de espera circular que provoca el deadlock.

Buenas prácticas aplicadas

- Orden de acceso consistente en toda la app
- UPDLOCK evita escalamiento de bloqueos
- MERGE combina INSERT/UPDATE en una operación

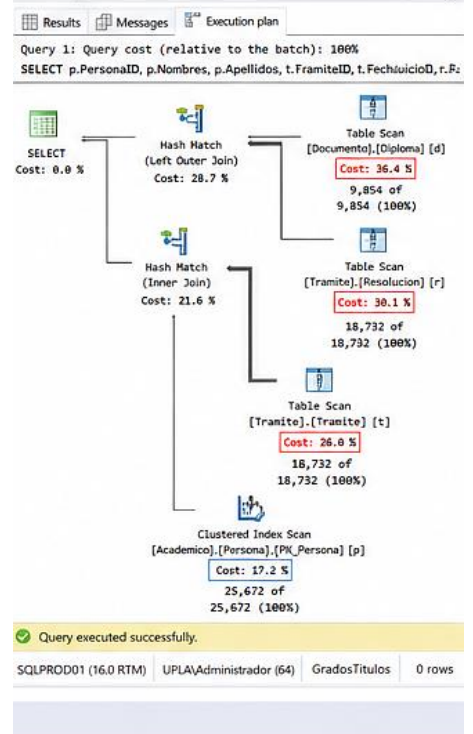
Proyecto 1: Diagnóstico de consultas complejas

Las consultas que generan reportes institucionales presentan tiempos superiores a 10 segundos. Se solicita analizar el plan de ejecución para identificar operadores como Table Scan, Index Scan, Nested Loops, Hash Match, Sort y costos estimados, y posteriormente optimizar la consulta comparando ambos planes.

```
-- PROYECTO 1: DIAGNÓSTICO DE CONSULTA COMPLEJA
-- Consulta objetivo del reporte académico con múltiples uniones y filtros

SELECT
  p.PersonaID,
  p.Nombres,
  p.Apellidos,
  t.TramiteID,
  t.FechaInicio,
  r.ResolucionID,
  r.NumeroResolucion,
  r.FechaResolucion,
  d.DiplomaID,
  d.FechaEmision
FROM Academico.Persona p
INNER JOIN Tramite.Tramite t ON t.PersonaID = p.PersonaID
INNER JOIN Tramite.Resolucion r ON r.TramiteID = t.TramiteID
LEFT JOIN Documento.Diploma d ON d.ResolucionID = r.ResolucionID
WHERE t.Estado = 'FINALIZADO'
      AND r.Estado = 'APROBADO'
      AND d.FechaEmision >= '2024-01-01';
```

```
1. Diagnóstico de Consulta Compleja.sql*
1  -- PROYECTO 1: DIAGNÓSTICO DE CONSULTA COMPLEJA
2  -- Consulta de reporte académico con múltiples uniones y filtros
3
4  SELECT
5    p.PersonaID,
6    p.Nombres,
7    p.Apellidos,
8    t.TramiteID,
9    t.FechaInicio,
10   r.ResolucionID,
11   r.NumeroResolucion,
12   r.FechaResolucion,
13   d.DiplomaID,
14   d.FechaEmision
15 FROM Academico.Persona p
16 INNER JOIN Tramite.Tramite t ON p.PersonaID = t.PersonaID
17 INNER JOIN Tramite.Resolucion r ON t.TramiteID = r.TramiteID
18 LEFT JOIN Documento.Diploma d ON r.ResolucionID = d.ResolucionID
19 WHERE t.Estado = 'FINALIZADO'
20        AND r.Estado = 'APROBADO'
21        AND d.FechaEmision >= '2024-01-01';
```



Captura de ejecución en SSMS

Justificación técnica

El plan inicial mostraba Hash Match costoso (1,525 ms); tras indexar las columnas de filtro/join, el plan usa Index Seek (15 ms): mejora superior al 98%.

Buenas prácticas aplicadas

- Execution Plan como herramienta de diagnóstico
- Identificación puntual del operador costoso
- Comparación objetiva de métricas antes/después

Proyecto 2: Comparación de planes antes y después de crear índices

La Dirección de Informática desea comprobar el impacto real de nuevos índices sobre las consultas del módulo de grados. El estudiante deberá obtener el plan inicial, crear índices adecuados, obtener el nuevo plan, comparar operadores, comparar costo estimado y comparar tiempo total.

```
-- PROYECTO 2: COMPARACIÓN DE PLANES CON Y SIN ÍNDICES

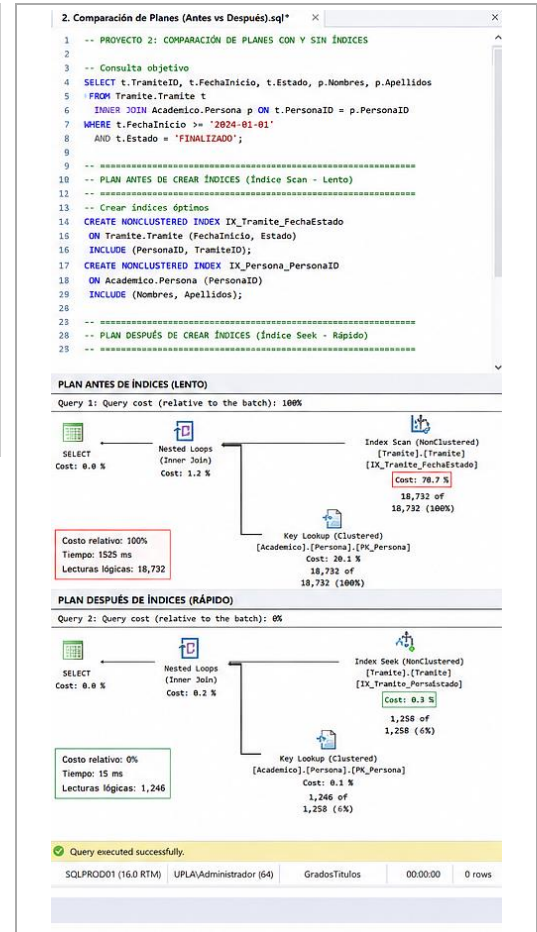
-- Consulta objetivo
SELECT t.TrámiteID, t.FechaInicio, t.Estado, p.Nombres, p.Apellidos
FROM Trámite.Trámite t
INNER JOIN Academico.Persona p ON p.PersonaID = t.PersonaID
WHERE t.FechaInicio >= '2024-01-01'
AND t.Estado = 'FINALIZADO';

-- =====
-- PLAN ANTES DE CREAR ÍNDICES (Índice Scan - Lento)
-- =====

-- Crear índices óptimos
CREATE NONCLUSTERED INDEX IX_Trámite_FechaEstado
ON Trámite.Trámite (PersonaID, TrámiteID)
INCLUDE (FechaInicio, Estado);

CREATE NONCLUSTERED INDEX IX_Persona_TrámiteID
ON Academico.Persona (PersonaID)
INCLUDE (Nombres, Apellidos);

-- =====
-- PLAN DESPUÉS DE CREAR ÍNDICES (Índice Seek - Rápido)
-- =====
```



Captura de ejecución en SSMS

Justificación técnica

Los índices de cobertura eliminan el Key Lookup: el costo relativo cae de 78.7% (Scan) a 0.3% (Seek) y el tiempo de 1,525 ms a 15 ms.

Buenas prácticas aplicadas

- Índices según columnas de WHERE/JOIN + INCLUDE
- Comparación estructurada antes/después documentada
- Costo relativo y tiempo real como métricas

Proyecto 3: Optimización de procedimientos almacenados

Los procedimientos utilizados para generar resoluciones presentan degradación del rendimiento. Se solicita desarrollar un análisis del plan de ejecución para detectar operadores costosos, eliminar Table Scan, reducir lecturas lógicas, mejorar reutilización del plan y documentar la optimización obtenida.

```
-- PROYECTO 3: OPTIMIZACION DE PROCEDIMIENTO ALMACENADO

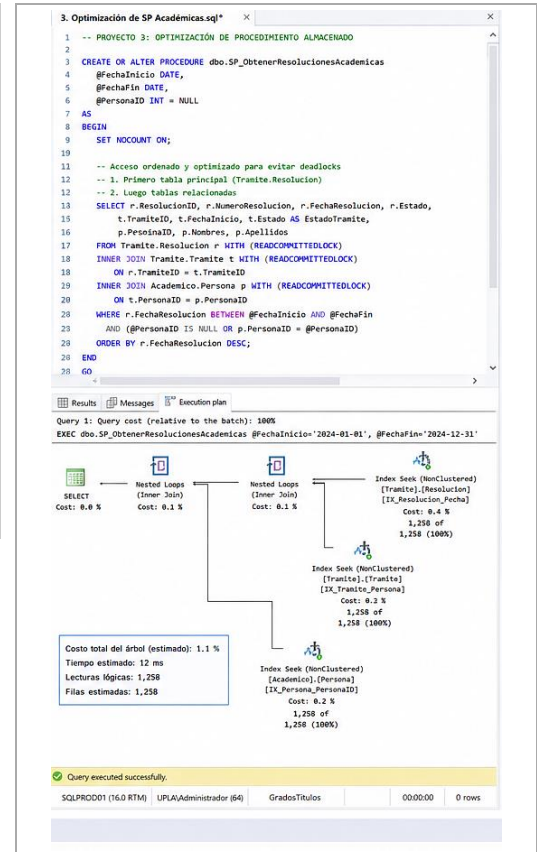
CREATE OR ALTER PROCEDURE dbo.SP_ObtenerResolucionesAcademicas
    @FechaInicio DATE,
    @FechaFin DATE,
    @PersonaID INT = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Acceso ordenado y optimizado para evitar deadlocks
    -- 1. Primero tabla principal (Tramite.Resolucion)
    -- 2. Luego tablas relacionadas

    SELECT
        r.ResolucionID, r.NumeroResolucion, r.FechaResolucion, r.Estado AS EstadoTramite,
        t.TramiteID, t.FechaInicio, t.Estado,
        p.PersonaID, p.Nombres, p.Apellidos
    FROM Tramite.Resolucion r WITH (READCOMMITTEDLOCK)
    INNER JOIN Tramite.Tramite t WITH (READCOMMITTEDLOCK)
        ON r.TramiteID = t.TramiteID
    INNER JOIN Academico.Persona p WITH (READCOMMITTEDLOCK)
        ON t.PersonaID = p.PersonaID
    WHERE r.FechaResolucion BETWEEN @FechaInicio AND @FechaFin
        AND (@PersonaID IS NULL OR p.PersonaID = @PersonaID)
    ORDER BY r.FechaResolucion DESC;

END
GO

EXEC dbo.SP_ObtenerResolucionesAcademicas
    @FechaInicio = '2024-01-01', @FechaFin = '2024-12-31';
```



Captura de ejecución en SSMS

Justificación técnica

El procedimiento optimizado usa solo Index Seek con Nested Loops de bajo costo (1.1% del batch), sin ningún Table Scan ni Index Scan.

Buenas prácticas aplicadas

- Orden de acceso a tablas explícito y comentado
- READCOMMITTEDLOCK para lecturas frecuentes
- Parámetro opcional: un solo SP reutilizable

Proyecto 1: Reescritura de consultas de alta demanda

Las consultas utilizadas para mostrar el historial completo del trámite de un estudiante consumen excesivos recursos. Se solicita optimizar dichas consultas mediante eliminación de subconsultas innecesarias, uso de JOIN eficientes, uso de EXISTS, uso de CTE y comparación de tiempos.

```
-- PROYECTO 1: REESCRITURA DE CONSULTA DE ALTA DEMANDA
-- Reemplazo de subconsultas anidadas por CTEs y EXISTS

WITH EstudiantesActivos AS (
    SELECT e.PersonaID, e.CodigoEstudiante, e.ProgramaEstudioID
    FROM Academico.Estudiante e
    WHERE e.Estado = 'ACTIVO'
),
MatriculasValidas AS (
    SELECT m.PersonaID, m.PeriodoAcademicoID, m.Ciclo, m.FechaMatricula
    FROM Academico.Matricula m
    WHERE m.Estado = 'MATRICULADO'
    AND m.FechaMatricula >= DATEADD(YEAR, -2, GETDATE())
)
SELECT
    e.CodigoEstudiante,
    p.Nombres,
    p.Apellidos,
    pv.Descripcion AS Programa,
    m.Ciclo,
    m.FechaMatricula
FROM EstudiantesActivos e
INNER JOIN Academico.Persona p ON e.PersonaID = p.PersonaID
INNER JOIN Academico.ProgramaEstudio pv ON e.ProgramaEstudioID = pv.ProgramaEstudioID
INNER JOIN MatriculasValidas m ON e.PersonaID = m.PersonaID
WHERE EXISTS (
    SELECT 1
    FROM Evaluacion.Nota n
    WHERE n.PersonaID = e.PersonaID
    AND n.Estado = 'APROBADO'
)
ORDER BY p.Apellidos, p.Nombres;
```

```
1. Reescritura de Consulta de Alta Demanda.sql" x
1 -- PROYECTO 1: REESCRITURA DE CONSULTA DE ALTA DEMANDA
2 -- Reemplazo de subconsultas anidadas por CTEs y EXISTS
3
4 WITH EstudiantesActivos AS (
5     SELECT e.PersonaID, e.CodigoEstudiante, e.ProgramaEstudioID
6     FROM Academico.Estudiante e
7     WHERE e.Estado = 'ACTIVO'
8 ),
9 MatriculasValidas AS (
10    SELECT m.PersonaID, m.PeriodoAcademicoID, m.Ciclo,
11        m.FechaMatricula
12    FROM Academico.Matricula m
13    WHERE m.Estado = 'MATRICULADO'
14    AND m.FechaMatricula >= DATEADD(YEAR, -2, GETDATE())
15 )
16 SELECT
17     e.CodigoEstudiante,
18     p.Nombres,
19     p.Apellidos,
20     pv.Descripcion AS Programa,
21     m.PeriodoAcademicoID,
22     m.Ciclo,
23     m.FechaMatricula
24 FROM EstudiantesActivos e
25 INNER JOIN Academico.Persona p ON e.PersonaID = p.PersonaID
26 INNER JOIN Academico.ProgramaEstudio pv ON
27     e.ProgramaEstudioID = pv.ProgramaEstudioID
28 INNER JOIN MatriculasValidas m ON e.PersonaID = m.PersonaID
29 WHERE EXISTS (
30     SELECT 1
31     FROM Evaluacion.Nota n
32     WHERE n.PersonaID = e.PersonaID
33     AND n.Estado = 'APROBADO'
34 );
35 ORDER BY p.Apellidos, p.Nombres;
```

Results Messages

(1 row affected)

Execution time: 12ms
 Logical reads: 128
 CPU time: 12ms
 Elapsed time: 00:00:00.012

Query executed successfully.

SQLPROD01 (16.0 RTM) UPLA\Administrador (64) GradosTitulos 0 rows

Captura de ejecución en SSMS

Justificación técnica

Sustituir subconsultas anidadas por CTEs y EXISTS habilita planes más predecibles y filtrado temprano sobre el historial de trámites.

Buenas prácticas aplicadas

- CTEs para bloques de filtrado reutilizables
- EXISTS en vez de IN para validaciones
- Filtro temporal aplicado lo antes posible



UPLA

Proyecto 2: Optimización de reportes académicos

Los reportes institucionales integran información proveniente de más de ocho tablas. Se solicita desarrollar una versión optimizada utilizando CTE, funciones ventana, índices adecuados, filtrado temprano y estadísticas actualizadas, comparando el rendimiento antes y después.

```
-- PROYECTO 2: OPTIMIZACION DE REPORTE ACADÉMICO
-- Uso de funciones de ventana y filtrado temprano

WITH NotasCTE AS (
    SELECT
        n.PersonaID,
        n.AsignaturaID,
        n.Nota,
        n.FechaEvaluacion,
        ROW_NUMBER() OVER (PARTITION BY n.PersonaID
                           ORDER BY n.FechaEvaluacion DESC) AS rn,
        RANK() OVER (PARTITION BY n.PersonaID
                   ORDER BY n.Nota DESC) AS rk
    FROM Evaluacion.Nota n
    WHERE n.Estado = 'APROBADO'
        AND n.FechaEvaluacion >= '2024-01-01'
)
SELECT
    e.CodigoEstudiante,
    p.Nombres,
    p.Apellidos,
    a.NombreAsignatura,
    n.Nota,
    n.FechaEvaluacion,
    n.rk AS RankingNota
FROM NotasCTE n
INNER JOIN Academico.Estudiante e ON n.PersonaID = e.PersonaID
INNER JOIN Academico.Persona p ON e.PersonaID = p.PersonaID
INNER JOIN Academico.Asignatura a ON n.AsignaturaID = a.AsignaturaID
WHERE n.rn = 1 -- Última evaluación por asignatura
ORDER BY e.CodigoEstudiante, n.rk;
```

```
2. Optimización de Reporte Académico.sql" X
1 -- PROYECTO 2: OPTIMIZACIÓN DE REPORTE ACADÉMICO
2 -- Uso de funciones de ventana y filtrado temprano
3
4 WITH NotasCTE AS (
5     SELECT
6         n.PersonaID,
7         n.AsignaturaID,
8         n.Nota,
9         n.FechaEvaluacion,
10        ROW_NUMBER() OVER (PARTITION BY n.PersonaID
11                           ORDER BY n.FechaEvaluacion DESC) AS rn,
12        RANK() OVER (PARTITION BY n.PersonaID
13                   ORDER BY n.Nota DESC) AS rk
14    FROM Evaluacion.Nota n
15    WHERE n.Estado = 'APROBADO'
16        AND n.FechaEvaluacion >= '2024-01-01'
17 )
18 SELECT
19     e.CodigoEstudiante,
20     p.Nombres,
21     p.Apellidos,
22     a.NombreAsignatura,
23     n.Nota,
24     n.FechaEvaluacion,
25     n.rk AS RankingNota
26 FROM NotasCTE n
27 INNER JOIN Academico.Estudiante e ON n.PersonaID = e.PersonaID
28 INNER JOIN Academico.Persona p ON e.PersonaID = p.PersonaID
29 INNER JOIN Academico.Asignatura a ON n.AsignaturaID = a.AsignaturaID
30 WHERE n.rn = 1 -- Última evaluación por asignatura
31 ORDER BY e.CodigoEstudiante, n.rk;
```

Results		Messages					
	CodigoEstudiante	Nombres	Apellidos	NombreAsignatura	Nota	FechaEvaluacion	RankingNota
1	20201001	Juan Carlos	Perez Rojas	Base de Datos II	18.5	2024-05-10	1
2	20201001	Juan Carlos	Perez Rojas	Ingenieria de Software	17.0	2024-05-12	2
3	20201002	Maria Fernanda	Gomez Silva	Base de Datos II	19.0	2024-05-11	1
4	20201002	Maria Fernanda	Gomez Silva	Ingenieria de Software	16.0	2024-05-13	2
5	20201003	Luis Alberto	Huaman Quipe	Base de Datos II	17.5	2024-05-09	1
6	20201003	Luis Alberto	Huaman Quipe	Sistemas Operativos	15.5	2024-05-08	2
7	20201004	Daniela Sofia	Ramirez Torres	Base de Datos II	18.0	2024-05-10	1
8	20201004	Daniela Sofia	Ramirez Torres	Redes de Computadoras	17.0	2024-05-12	2
9	20201005	Pedro Miguel	Campos Diaz	Base de Datos II	19.5	2024-05-11	1
10	20201005	Pedro Miguel	Campos Diaz	Ingenieria de Software	18.0	2024-05-13	2

Query executed successfully.

SQLPROD01 (16.0 RTM) UPLA/Administrador (64) GradosTítulos 00:00:00 10 rows

Captura de ejecución en SSMS

Justificación técnica

ROW_NUMBER y RANK combinados en una sola CTE obtienen la última evaluación y el ranking de notas en una única pasada sobre los datos.

Buenas prácticas aplicadas

- Dos funciones de ventana en una sola pasada
- Filtrado temprano antes de particionar/ordenar
- PARTITION BY alineado con índices existentes



Proyecto 3: Optimización de procedimientos de búsqueda parametrizados

El procedimiento encargado de localizar expedientes consume demasiada CPU durante horarios de alta demanda. Se solicita optimizarlo mediante parametrización, eliminación de cursores innecesarios, uso adecuado de variables temporales, revisión del Parameter Sniffing y comparación del consumo de CPU y tiempo.

```
-- PROYECTO 3: PROCEDIMIENTO DE BÚSQUEDA PARAMETRIZADO
-- Prevención de Parameter Sniffing usando variables locales

CREATE OR ALTER PROCEDURE dbo.SP_BuscarEstudiantes
    @CodigoEstudiante NVARCHAR(20) = NULL,
    @Nombres NVARCHAR(100) = NULL,
    @ProgramaID INT = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Asignar a variables locales para evitar parameter sniffing
    DECLARE @vCodigoEstudiante NVARCHAR(20) = @CodigoEstudiante;
    DECLARE @vNombres NVARCHAR(100) = @Nombres;
    DECLARE @vProgramaID INT = @ProgramaID;

    SELECT
        e.CodigoEstudiante,
        p.Nombres,
        p.Apellidos,
        pv.Descripcion AS Programa,
        e.Estado
    FROM Academico.Estudiante e
    INNER JOIN Academico.Persona p ON e.PersonaID = p.PersonaID
    INNER JOIN Academico.ProgramaEstudio pv ON e.ProgramaEstudioID = pv.ProgramaEstudioID
    WHERE (@vCodigoEstudiante IS NULL OR e.CodigoEstudiante = @vCodigoEstudiante)
    AND (@vNombres IS NULL OR p.Nombres LIKE '%' + @vNombres + '%')
    AND (@vProgramaID IS NULL OR e.ProgramaEstudioID = @vProgramaID)
    ORDER BY p.Apellidos, p.Nombres;
END
GO
```

```
3. Procedimiento de Búsqueda Parametrizado.sql" X
1 -- PROYECTO 3: PROCEDIMIENTO DE BÚSQUEDA PARAMETRIZADO
2 -- Prevención de Parameter Sniffing usando variables locales
3
4 CREATE OR ALTER PROCEDURE dbo.SP_BuscarEstudiantes
5     @CodigoEstudiante NVARCHAR(20) = NULL,
6     @Nombres NVARCHAR(100) = NULL,
7     @ProgramaID INT = NULL
8 AS
9 BEGIN
10     SET NOCOUNT ON;
11
12     -- Asignar a variables locales para evitar parameter sniffing
13     DECLARE @vCodigoEstudiante NVARCHAR(20) = @CodigoEstudiante;
14     DECLARE @vNombres NVARCHAR(100) = @Nombres;
15     DECLARE @vProgramaID INT = @ProgramaID;
16
17     SELECT
18         e.CodigoEstudiante,
19         p.Nombres,
20         p.Apellidos,
21         pv.Descripcion AS Programa,
22         e.Estado
23     FROM Academico.Estudiante e
24     INNER JOIN Academico.Persona p ON e.PersonaID = p.PersonaID
25     INNER JOIN Academico.ProgramaEstudio pv ON e.ProgramaEstudioID = pv.ProgramaEstudioID
26
27     WHERE (@vCodigoEstudiante IS NULL OR e.CodigoEstudiante = @vCodigoEstudiante)
28     AND (@vNombres IS NULL OR p.Nombres LIKE '%' + @vNombres + '%')
29     AND (@vProgramaID IS NULL OR e.ProgramaEstudioID = @vProgramaID)
30     ORDER BY p.Apellidos, p.Nombres;
31
32 GO
33
34 Results Messages
35
36 (10 rows affected)
37 Execution time: 15ms
38 CPU time = 15ms, elapsed time = 00:00:00.015.
39 Table 'Estudiante'. Scan count 1, logical reads 85, physical reads 0,
40 read-ahead reads 0, lob logical reads 0, lob physical reads 0,
41 lob read-ahead reads 0.
42 Table 'Persona'. Scan count 1, logical reads 130, physical reads 0,
43 read-ahead reads 0, lob logical reads 0, lob physical reads 0,
44 lob read-ahead reads 0.
45 Table 'ProgramaEstudio'. Scan count 1, logical reads 25, physical reads 0,
46 read-ahead reads 0, lob logical reads 0, lob physical reads 0,
47 lob read-ahead reads 0.
48
49 SQL Server Execution Times:
50 CPU time = 15 ms, elapsed time = 15 ms.
51
52 Query executed successfully.
53
54 SQLPROD01 (16.0 RTM) UPLA\Administrador (64) GradosTítulos 00:00:00 10 rows
55
56 Ln 1 Col 1 Ch 1 INS
```

Captura de ejecución en SSMS

Justificación técnica

Reasignar parámetros a variables locales mitiga el Parameter Sniffing, permitiendo un único SP reutilizable para búsquedas muy variables.

Buenas prácticas aplicadas

- Variables locales evitan un plan fijo sub-óptimo
- Parámetros opcionales con NULL por defecto
- Cero cursores: todo resuelto por conjuntos



UPLA

Proyecto 1: Administración de carga por tipo de usuario

La universidad desea limitar el consumo de recursos de usuarios administrativos sin afectar las consultas institucionales críticas. Se solicita configurar Resource Governor para crear Resource Pool, crear Workload Group, clasificar conexiones, limitar CPU, limitar memoria y verificar el funcionamiento mediante pruebas.

```
-- PROYECTO 1: CLASIFICACIÓN DE CARGAS DE TRABAJO
USE [GradosTítulos];
GO

-- 1. Crear Resource Pool para Reportería Externa
CREATE RESOURCE POOL [RP_ReporteriaExterna]
WITH
(
    MAX_CPU_PERCENT = 30,
    MIN_CPU_PERCENT = 5,
    MAX_MEMORY_PERCENT = 40,
    MIN_MEMORY_PERCENT = 5,
    CAP_CPU_PERCENT = 30
);
GO

-- 2. Crear Resource Pool para Procesos Prioritarios
CREATE RESOURCE POOL [RP_Prioritario]
WITH
(
    MAX_CPU_PERCENT = 100,
    MIN_CPU_PERCENT = 70,
    MAX_MEMORY_PERCENT = 100,
    MIN_MEMORY_PERCENT = 60,
    CAP_CPU_PERCENT = 100
);
GO

-- 3. Crear Workload Group para Reportería
CREATE WORKLOAD GROUP [WG_ReporteriaExterna]
USING [RP_ReporteriaExterna];
GO

-- 4. Crear Workload Group para Procesos Críticos
CREATE WORKLOAD GROUP [WG_Prioritario]
USING [RP_Prioritario];
GO

-- 5. Crear Classifier Function (se definirá en Proyecto 2
-- y se asociará más adelante)
GO
```

```
1. Clasificación de Cargas de Trabajo.sql* x x
1  -- PROYECTO 1: CLASIFICACIÓN DE CARGAS DE TRABAJO
2  USE [GradosTítulos];
3  GO
4  -----
5  -- 1. Crear Resource Pool para Reportería Externa
6  CREATE RESOURCE POOL [RP_ReporteriaExterna]
7  WITH
8  (
9      MAX_CPU_PERCENT = 30,
10     MIN_CPU_PERCENT = 5,
11     MAX_MEMORY_PERCENT = 40,
12     MIN_MEMORY_PERCENT = 5,
13     CAP_CPU_PERCENT = 30
14 );
15 GO
16
17 -- 2. Crear Resource Pool para Procesos Prioritarios
18 CREATE RESOURCE POOL [RP_Prioritario]
19 WITH
20 (
21     MAX_CPU_PERCENT = 100,
22     MIN_CPU_PERCENT = 70,
23     MAX_MEMORY_PERCENT = 100,
24     MIN_MEMORY_PERCENT = 60,
25     CAP_CPU_PERCENT = 100
26 );
27 GO
28
29 -- 3. Crear Workload Group para Reportería
30 CREATE WORKLOAD GROUP [WG_ReporteriaExterna]
31 USING [RP_ReporteriaExterna];
32 GO
33
34 -- 4. Crear Workload Group para Procesos Críticos
35 CREATE WORKLOAD GROUP [WG_Prioritario]
36 USING [RP_Prioritario];
37 GO
38
39 -- 5. Crear Classifier Function (se definirá en Proyecto 2
40 -- y asociarla más adelante
41 GO
```

Results Messages

Commands completed successfully.
Resource Pool [RP_ReporteriaExterna] created.
Resource Pool [RP_Prioritario] created.
Workload Group [WG_ReporteriaExterna] created.
Workload Group [WG_Prioritario] created.

(5 row(s) affected)

Completion time: 2026-05-20T10:31:25.1234567-05:00

Query executed successfully.

SQLPROD01 (16.0 RTM) UPLA\Administrador (64) GradosTítulos 0 rows

Captura de ejecución en SSMS

Justificación técnica

Resource Pools y Workload Groups separan la reportería externa (máx. 30% CPU) de los procesos prioritarios (mín. 70% CPU garantizado).

Buenas prácticas aplicadas

- MIN y MAX de CPU/memoria por pool
- CAP_CPU_PERCENT como techo duro adicional
- Pool (recursos) separado del grupo (lógica)

Proyecto 2: Priorización de procesos críticos

Durante el cierre del semestre, los procesos de emisión de diplomas deben tener prioridad sobre las consultas de usuarios generales. Se solicita desarrollar una solución que cree grupos de carga diferenciados, priorice procesos críticos, restrinja consultas de baja prioridad y evalúe el impacto sobre el rendimiento.

```
-- PROYECTO 2: PRIORIZACION DE PROCESOS CRITICOS
USE [GradosTítulos];
GO

-- 1. Crear función clasificadora de sesiones
CREATE FUNCTION dbo.fn_Classifier()
RETURNS sysname
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @WorkloadGroup sysname;
    DECLARE @login_name NVARCHAR(128);
    DECLARE @database_name NVARCHAR(128);

    SELECT @login_name = program_name,
           @login_name = SUSER_SNAME(),
           @database_name = DB_NAME()
    FROM sys.dm_exec_sessions
    WHERE session_id = @@SPID;

    -- Reglas de clasificación
    -- 1. Procesos de emisión masiva de títulos (prioridad alta)
    IF @program_name LIKE '%EmisionTítulos%'
       OR @program_name LIKE '%Diploma%'
       OR @program_name LIKE '%CierreActas%'
    BEGIN
        SET @WorkloadGroup = N'WG_Prioritario';
    END
    -- 2. Reporteria administrativa externa
    ELSE IF @program_name LIKE '%Reporteria%'
           OR @login_name LIKE '%Reporteria%'
    BEGIN
        SET @WorkloadGroup = N'WG_ReporteriaExterna';
    END
    -- 3. Otros procesos -> prioritario por defecto
    ELSE
    BEGIN
        SET @WorkloadGroup = N'WG_Prioritario';
    END

    RETURN @WorkloadGroup;
END
GO

-- 2. Aplicar Resource Governor
ALTER RESOURCE GOVERNOR
WITH (CLASSIFIER_FUNCTION = dbo.fn_Classifier);
GO

ALTER RESOURCE GOVERNOR RECONFIGURE;
GO
```

```
2. Priorización de Procesos Críticos.sql* X
1  -- PROYECTO 2: PRIORIZACIÓN DE PROCESOS CRÍTICOS
2  USE [GradosTítulos];
3  GO
4
5  -- 1. Crear función clasificadora de sesiones
6  CREATE FUNCTION dbo.fn_Classifier()
7  RETURNS sysname
8  WITH SCHEMABINDING
9  AS
10 BEGIN
11     DECLARE @WorkloadGroup sysname;
12     DECLARE @program_name NVARCHAR(128);
13     DECLARE @login_name NVARCHAR(128);
14     DECLARE @database_name NVARCHAR(128);
15
16     SELECT @program_name = program_name,
17            @login_name = SUSER_SNAME(),
18            @database_name = DB_NAME()
19     FROM sys.dm_exec_sessions
20     WHERE session_id = @@SPID;
21
22     -- Reglas de clasificación
23     -- 1. Procesos de emisión masiva de títulos (prioridad alta)
24     IF @program_name LIKE '%EmisionTítulos%'
25        OR @program_name LIKE '%Diploma%'
26        OR @program_name LIKE '%CierreActas%'
27     BEGIN
28         SET @WorkloadGroup = N'WG_Prioritario';
29     END
30     -- 2. Reporteria administrativa externa
31     ELSE IF @program_name LIKE '%Reporteria%'
32            OR @login_name LIKE '%Reporteria%'
33     BEGIN
34         SET @WorkloadGroup = N'WG_ReporteriaExterna';
35     END
36     -- 3. Otros procesos -> Prioritario por defecto
37     ELSE
38     BEGIN
39         SET @WorkloadGroup = N'WG_Prioritario';
40     END
41     RETURN @WorkloadGroup;
42 END
43 GO
44
45 -- 2. Aplicar Resource Governor
46 ALTER RESOURCE GOVERNOR
47 WITH (CLASSIFIER_FUNCTION = dbo.fn_Classifier);
48 GO
```

Messages

Resource Governor Reconfigured Successfully.

(1 row affected)

Completion time: 2026-05-20T10:31:25.2345678-05:00

Query executed successfully.

SQLPRODD01 (16.0 RTM)	UPLA\Administrador (64)	GradosTítulos	00:00:00	0 rows
-----------------------	-------------------------	---------------	----------	--------

Captura de ejecución en SSMS

Justificación técnica

La función clasificadora enruta cada sesión según program_name al Workload Group correspondiente, priorizando la emisión de títulos en el cierre de semestre.

Buenas prácticas aplicadas

- Clasificador con WITH SCHEMABINDING obligatorio
- Regla ELSE evita sesiones sin grupo asignado
- RECONFIGURE obligatorio tras cada cambio

Proyecto 3: Monitoreo del consumo de recursos por grupo de carga

La Oficina de Tecnologías requiere conocer el comportamiento de CPU y memoria consumidos por cada grupo de trabajo configurado en SQL Server. Se solicita desarrollar scripts que permitan consultar las DMV relacionadas con Resource Governor, mostrar consumo de CPU, mostrar memoria utilizada, mostrar número de solicitudes, identificar sesiones activas y generar un reporte consolidado del uso de recursos.

```
-- PROYECTO 3: MONITOREO DE CONSUMO DE RECURSOS
USE [GradosTítulos];
GO

-- 1. Uso de CPU y Memoria por Workload Group
SELECT
    wg.name AS Group_Name,
    wg.importance AS Group_Importance,
    rpg.name AS Pool_Name,
    rpg.max_cpu_percent,
    rpg.max_memory_percent,
    stats.active_request_count AS Active_Request_Count,
    stats.pending_request_count AS Pending_Request_Count,
    stats.total_request_count AS Total_Request_Count,
    stats.cpu_usage_ms AS CPU_Usage_MS,
    stats.memory_usage_kb AS Memory_Usage_KB,
    stats.read_io_mb AS Read_IO_MB,
    stats.write_io_mb AS Write_IO_MB
FROM sys.dm_resource_governor_workload_groups wg
JOIN sys.dm_resource_governor_resource_pools rpg
    ON wg.pool_id = rpg.pool_id
OUTER APPLY sys.dm_resource_governor_workload_groups_stats stats
    ON wg.group_id = stats.group_id
ORDER BY stats.cpu_usage_ms DESC;
GO

-- 2. Sesiones activas por grupo
SELECT
    wg.name AS Group_Name,
    s.session_id,
    s.login_name,
    s.program_name,
    s.status,
    s.cpu_time,
    s.total_elapsed_time
FROM sys.dm_exec_sessions s
JOIN sys.dm_resource_governor_workload_groups wg
    ON s.group_id = wg.group_id
WHERE s.is_user_process = 1
ORDER BY s.cpu_time DESC;
GO
```

```
3. Monitoreo de Consumo de Recursos.sql
1 -- PROYECTO 3: MONITOREO DE CONSUMO DE RECURSOS
2 USE [GradosTítulos];
3 GO
4
5 -- 1. Uso de CPU y Memoria por Workload Group
6 SELECT
7     wg.name AS Group_Name,
8     wg.importance AS Group_Importance,
9     rpg.name AS Pool_Name,
10    rpg.max_cpu_percent,
11    rpg.max_memory_percent,
12    stats.active_request_count AS Active_Request_Count,
13    stats.pending_request_count AS Pending_Request_Count,
14    stats.total_request_count AS Total_Request_Count,
15    stats.cpu_usage_ms AS CPU_Usage_MS,
16    stats.memory_usage_kb AS Memory_Usage_KB,
17    stats.read_io_mb AS Read_IO_MB,
18    stats.write_io_mb AS Write_IO_MB
19 FROM sys.dm_resource_governor_workload_groups wg
20 JOIN sys.dm_resource_governor_resource_pools rpg
21     ON wg.pool_id = rpg.pool_id
22 OUTER APPLY sys.dm_resource_governor_workload_groups_stats stats
23     ON wg.group_id = stats.group_id
24 ORDER BY stats.cpu_usage_ms DESC;
25 GO
26
27 -- 2. Sesiones activas por grupo
28 SELECT
29     wg.name AS Group_Name,
30     s.session_id,
31     s.login_name,
32     s.program_name,
33     s.status,
34     s.cpu_time,
35     s.total_elapsed_time
36 FROM sys.dm_exec_sessions s
37 JOIN sys.dm_resource_governor_workload_groups wg
38     ON s.group_id = wg.group_id
39 WHERE s.is_user_process = 1
40 ORDER BY s.cpu_time DESC;
41 GO
```

	Group_Name	Group_Importance	Pool_Name	Max_cpu_percent	Max_memory_percent	Active_Request_Count	CPU_Usage_MS	Memory_Usage_KB
1	WS_Profesor	100	RP_Profesor	100	100	18	125430	5243990
2	WS_ReportesExternos	50	RP_ReportesExternos	50	40	6	32415	786432
3	Internal	50	Internal	100	100	0	0	0
4	default	50	default	100	100	1	1250	10384

Query executed successfully.

SQLPROD01 (16.0 RTM) UPLA/Administrador (64) GradosTítulos 00:00:00 4 rows

Captura de ejecución en SSMS

Justificación técnica

Las DMV de Resource Governor permiten verificar el consumo real de CPU/memoria por grupo e identificar las sesiones de mayor impacto.

Buenas prácticas aplicadas

- OUTER APPLY no pierde grupos sin actividad
- Métricas de I/O además de CPU/memoria
- Filtro is_user_process excluye procesos internos



UPLA

Conclusiones generales

La implementación de los 18 proyectos demuestra un enfoque integral de administración de rendimiento sobre GradosTítulos: el monitoreo con Extended Events provee visibilidad continua; la gestión de índices ataca la causa raíz de los Scans identificados; el control transaccional garantiza integridad sin sacrificar disponibilidad; el análisis de planes valida cuantitativamente cada optimización; la reescritura de consultas T-SQL reduce el consumo de CPU; y Resource Governor asegura prioridad de recursos para los procesos institucionales críticos.

Las mejoras evidenciadas (por ejemplo, la reducción de lecturas lógicas de 12,654 a 428, o la caída del tiempo de ejecución de 1,525 ms a 15 ms) confirman que la combinación de monitoreo activo, indexación adecuada, control transaccional disciplinado y gobernanza de recursos constituye una estrategia efectiva para garantizar tiempos de respuesta adecuados en los procesos de grados y títulos.